

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 1

Základy grafů

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>



ZS 2025/2026, 22. září 2025

(Verze dokumentu: 14. 12. 2025 19:21)

Obsah předmětu BI-AG1

- Základy teorie grafů.
- Základní grafové algoritmy (např. procházení grafu, hledání nejkratších cest, konstrukce kostry).
- Základní datové struktury.
- Základní algoritmy pro efektivní řešení některých důležitých úloh v informatice (např. řazení, vyhledávání, dynamické programování).

Cíle předmětu BI-AG1

Co si mají studenti z BI-AG1 odnést:

- Pochopení principů a myšlenek, na kterých je postavena
 - ▶ teorie grafů
 - ▶ teorie efektivních datových struktur a algoritmů.
- Získání zkušenosti, jak tyto principy a myšlenky tvůrčím způsobem využívat pro formulaci a efektivní řešení základních problémů v informatice (např. jazykem teorie grafů).
- Chápat algoritmy modulárně.

Znát definice pojmů je potřebné, ale pouze mechanické učení vět, jejich důkazů a algoritmů nestačí.

Doporučená literatura

- M. Mareš, T. Valla: *Průvodce labyrintem algoritmů*, CZ.NIC, Praha, 2017. knihy.nic.cz
- P. Kovář: [Diskrétní matematika a úvod do teorie grafů](#)
- Grafy a grafové algoritmy:
 - ▶ P. Hliněný: [Základy teorie grafů pro \(nejen\) informatiky](#)
 - ▶ J. Demel: *Grafy a jejich aplikace*. Vlastním nákladem, 2015.
 - ▶ J. Nešetřil, J. Matoušek: *Kapitoly z diskrétní matematiky*. Karolinum Praha, 2010.
 - ▶ ...
- Datové struktury a algoritmy:
 - ▶ T.H. Cormen, C.E. Leiserson, R.L. Rivest, C. Stein: *Introduction to Algorithms*. Third edition, MIT Press, 2009.
 - ▶ K. Mehlhorn, P. Sanders: *Algorithms and Data Structures: The Basic Toolbox*. Springer 2008.
 - ▶ ...

Jak najít nejkratší cestu v 2D bludišti?

- Uvažujme 2D bludiště B zadané $k \times \ell$ maticí políček, kde každé políčko $B[i, j]$ je buď **neoznačené volné** nebo **zed'**. V Bludišti se chodí pouze **vodorovně** nebo **svisle**.
- Dále je dáno **startovní** políčko s a **cílové** t .
- Úkolem je najít **nejkratší cestu** z s do t .
- Řešme nejprve úlohu najít **délku nejkratší cesty** z s do t .

s				
				t

Jak najít nejkratší cestu v 2D bludišti? – příklad

Řešením je **záplavový** (či též **vlnový**) algoritmus:

- Do startovního políčka s nalijeme „pravoúhlou“ vodu (ta teče pouze svisle nebo vodorovně).
- V čase 1 zaplaví voda **sousedy** políčka s (= vrcholy ve vzd. 1).
- V čase 2 zaplaví voda dosud nezaplavené **sousedy sousedů** (= vrcholy ve vzdálenosti 2), atd.

V následující animaci je do každého zaplaveného políčka zapsán **čas**, kdy do něj dorazila záplavová vlna, je-li s levé horní políčko.

0	1	2	3	4
	2			
4	3	4	5	6
5			6	7
				8

Neorientovaný graf

Definice

Neorientovaný graf je uspořádaná dvojice (V, E) , kde

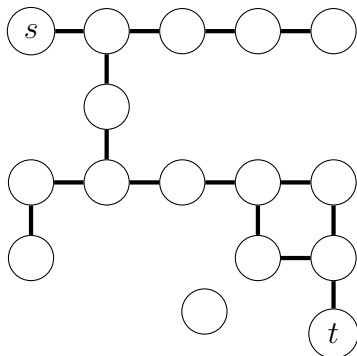
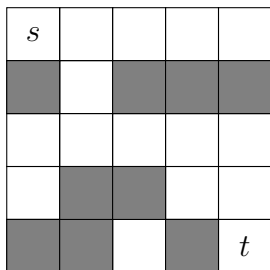
- V je neprázdná konečná množina **vrcholů**,
- E je množina **hran**.

Hrana je dvouprvková podmnožina V (čili neuspořádaná dvojice vrcholů), značíme $\{u, v\}$.

- Množinu všech dvouprvkových podmnožin množiny V (tedy všech možných hran) budeme označovat $\binom{V}{2}$.
- Obvykle budeme říkat jen **graf**, tj. neříkáme neorientovaný.

Znázornění grafu

- Pro větší názornost často používáme obrázky.
- Vrcholy znázorníme kolečky nebo puntíky.
- Hrany znázorníme spojením příslušných vrcholů čarou.



Značení

- Nechť G je graf. Pak
 - ▶ jeho množinu vrcholů a hran budeme značit $V(G)$ a $E(G)$,
 - ▶ počet jeho vrcholů a hran budeme obvykle značit $n = |V(G)|$ a $m = |E(G)|$.

Definice

Nechť $e = \{u, v\}$ je hrana v grafu G . Pak řekneme, že

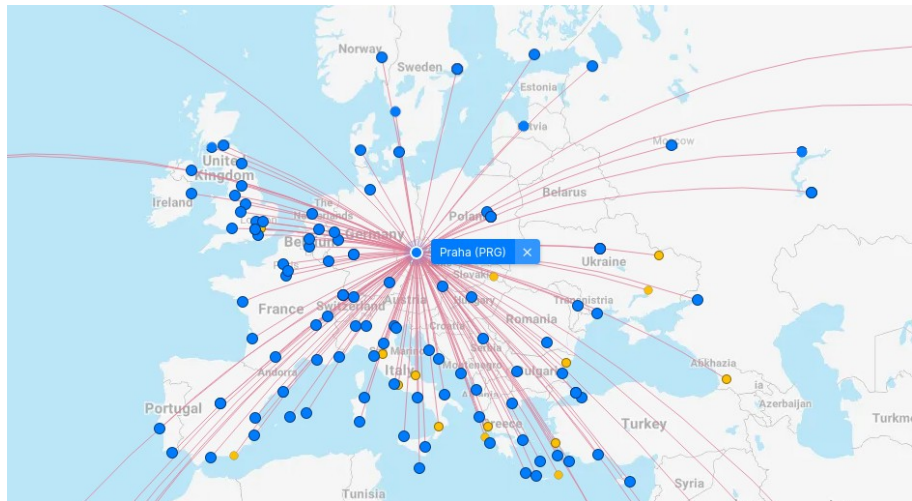
- vrcholy u a v jsou **koncové vrcholy** hrany e ,
- u je **sousedem** v v G (a naopak),
- u i v jsou **incidentní** s hranou e .

Grafy jsou všude!



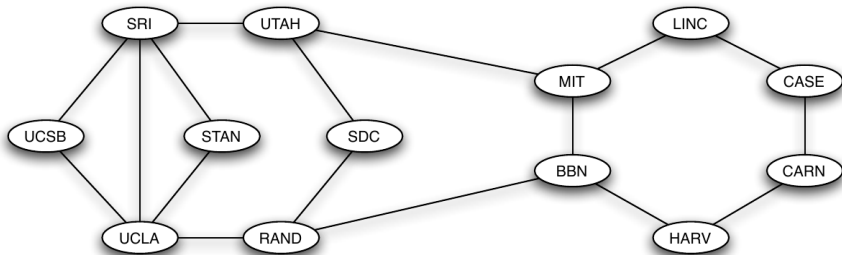
Zdroj: <https://www.dpp.cz/metro-d/popis-projektu/celkova-koncepce>

Grafy jsou všude!



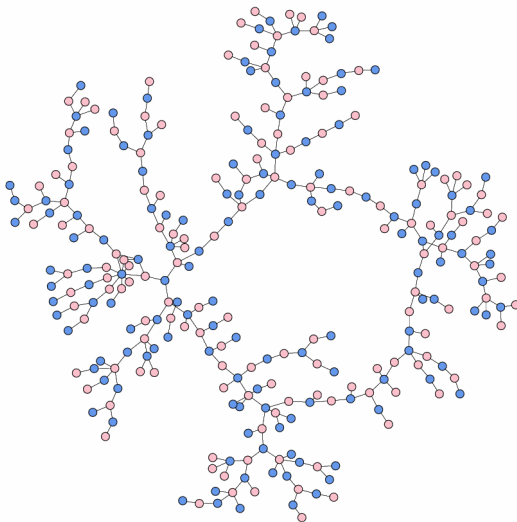
Zdroj: <https://www.flightconnections.com/>

Grafy jsou všude!



Zdroj: D. Easley, J. Kleinberg. Networks, Crowds, and Markets: Reasoning about a Highly Connected World

Grafy jsou všude!

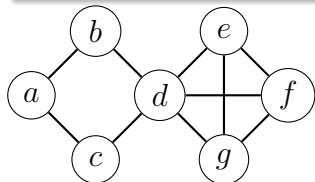


Zdroj: D. Easley, J. Kleinberg. Networks, Crowds, and Markets: Reasoning about a Highly Connected World

Sled a cesta v grafu

Definice

- **Sled délky k v grafu G** je sekvence $v_0, e_1, v_1, e_2, \dots, e_k, v_k$ taková, že $e_i = \{v_{i-1}, v_i\}$ a $e_i \in E(G)$ pro všechna $i = 1, \dots, k$.
- **Cesta v grafu G** je sled, ve kterém se neopakují vrcholy (a tedy ani hrany).
- Má-li cesta P v grafu G koncové vrcholy $s = v_0$ a $t = v_k$, mluvíme o **cestě z s do t** , nebo o **s - t -cestě**. (Připouštíme $s = t$, cesta tedy může mít nulovou délku).
- **Délka s - t -cesty** je počet hran v této cestě (v grafu G).
- **Vzdálenost s a t** $d(s, t)$ je délka nejkratší s - t -cesty.



- $a, \{a, b\}, b, \{b, d\}, d, \{d, e\}, e, \{e, g\}, g, \{g, d\}, d, \{d, f\}, f$ je sled ne cesta
- $a, \{a, b\}, b, \{b, d\}, d, \{d, f\}, f$ je sled i cesta

Breadth-First Search (BFS)

prohledávání grafu do šířky

Vstup

Neorientovaný graf $G = (V, E)$ a vrchol $s \in V$

Výstup

- Pole vzdáleností D takové, že

$$D[v] = \begin{cases} k & k \text{ je délka nejkratší } s\text{-}v\text{-cesty v } G \\ \text{undef.} & \text{neexistuje žádná } s\text{-}v\text{-cesta v } G \end{cases}$$

- Pole předchůdců P takové, že

$$P[v] = \begin{cases} w & w \text{ je před } v \text{ na nějaké nejkratší } s\text{-}v\text{-cestě v } G \\ \text{undef.} & \text{neexistuje žádná } s\text{-}v\text{-cesta v } G \end{cases}$$

Algoritmus **BFS**(graf G , vrchol s):

(1) Pro každý vrchol $v \in V(G)$:

(2) stav[v] := nenalezený

(3) D[v] := P[v] := undef

(4) Q := fronta obsahující jediný vrchol s

(5) stav[s] := otevřený

(6) D[s] := 0, P[s] := $\perp$

(7) Dokud je fronta Q neprázdná:

(8) Odeber z Q první vrchol, nechť to je v

(9) Pro všechny sousedy w vrcholu v :

(10) Pokud stav[w] = nenalezený:

(11) stav[w] := otevřený

(12) D[w] := D[v] + 1

(13) P[w] := v

(14) Přidej w na konec fronty Q

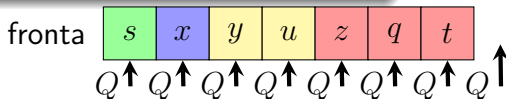
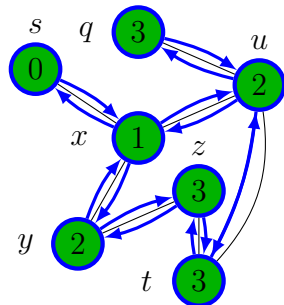
(15) stav[v] := uzavřený

(16) Vrať (D, P)

Algoritmus **BFS**(G, s):

- (1) Pro každý vrchol $v \in V(G)$:
- (2) stav[v] := nenalezený
- (3) D[v] := P[v] := undef
- (4) Q := fronta obsahující jediný vrchol s
- (5) stav[s] := otevřený
- (6) D[s] := 0, P[s] := $\perp$
- (7) Dokud je fronta Q neprázdná:
- (8) Vyjmi z Q první vrchol, nechť to je v
- (9) Pro všechny sousedy w vrcholu v :
- (10) Pokud stav[w] = nenalezený:
- (11) stav[w] := otevřený
- (12) D[w] := D[v] + 1
- (13) P[w] := v
- (14) Přidej w na konec fronty Q
- (15) stav[v] := uzavřený
- (16) Vrať (D, P)

t	z	u	
q	u		
s	x		
u	q	x	t
x	y	u	s
y	x	z	
z	y	t	



Otázky, které se v BI-AG1 budeme učit zodpovídat

- 1 Je pravda, že se algoritmus na každém (konečném) vstupu zastaví?
- 2 Jak formálně specifikovat požadavky na vstup a výstup?
- 3 Pracuje uvedený algoritmus správně?
- 4 Jak poznat, že řešení neexistuje?
- 5 Jak lze algoritmus efektivně realizovat?

Konečnost a správnost algoritmu BFS

Věta (o konečnosti algoritmu BFS)

Algoritmus **BFS**(G, s) se vždy zastaví.

Pokud víme, že se algoritmus **BFS**(G, s) **vždy zastaví**, můžeme (a musíme) ukázat, že vždy vydá **správný** (očekávaný) výsledek.
To znamená dokázat jeho 3 vlastnosti.

Věta (o správnosti algoritmu BFS)

Vlastnost 1: Po skončení **BFS**(G, s) jsou uzavřené právě ty vrcholy, do kterých vede cesta ze startu s a ostatní vrcholy zůstanou nenalezené.

Vlastnost 2: Pro všechny uzavřené vrcholy v platí $D[v] = d(s, v)$ = délka nejkratší cesty ze startu s do vrcholu v .

Vlastnost 3: Pro všechny uzavřené vrcholy v platí $P[v] = w$, kde w je předchůdce v na nějaké nejkratší cestě ze startu s do vrcholu v .

(na cvičeních)

Důkaz správnosti algoritmu BFS I

Věta (o konečnosti algoritmu BFS)

Algoritmus **BFS**(G, s) se vždy zastaví.

Důkaz.

- Každý vrchol je přidán do fronty Q nejvýše jednou, neboť je předtím otevřen a nemůže už tedy vícekrát splnit podmínku na řádku (10), že je nenalezený.
- V každé iteraci cyklu (7)–(15) je jeden vrchol odebrán z fronty.
- Algoritmus se tedy zastaví nejvýše po $|V(G)|$ iteracích cyklu (7)–(15). □

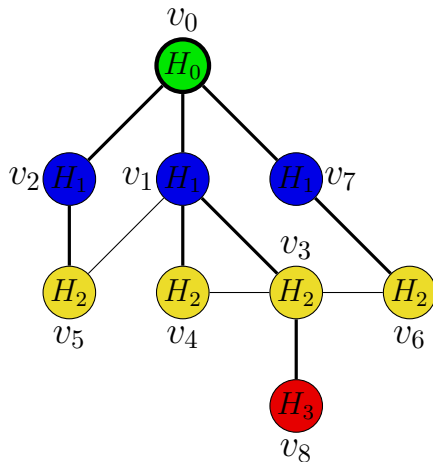
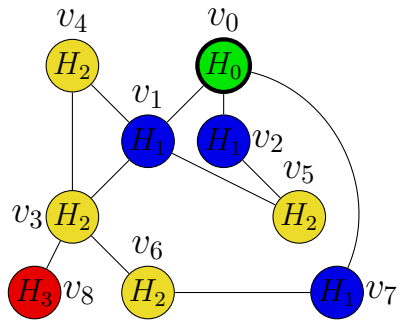
BFS fáze a hladiny

Před dokončením důkazu správnosti definujeme **BFS fáze a hladiny**.

Definice (BFS fáze F_i a BFS hladiny H_i)

- Hladina $H_0 = \{s\}$.
- Fáze F_0 trvá od odebrání s z Q , otevření a vložení všech jeho sousedů do Q až do uzavření s .
- Hladina H_i je množina všech vrcholů otevřených a vložených do Q ve fázi F_{i-1} .
- Fáze F_i trvá od konce fáze F_{i-1} až po uzavření všech vrcholů z H_i (zahrnuje tedy vybrání všech vrcholů z H_i z Q , otevření jejich dosud nenalezených sousedů a vložení těchto sousedů do Q).
- Necht h je číslo poslední fáze $\text{BFS}(G, s)$ (po ní se $\text{BFS}(G, s)$ zastaví).

Příklad hladin v grafu



Důkaz správnosti algoritmu BFS I

Vlastnost 1: Po skončení $\text{BFS}(G, s)$ jsou uzavřené právě ty vrcholy, do kterých vede cesta ze startu s a ostatní vrcholy zůstanou nenalezené.

Důkaz.

- Prvním otevřeným a následně uzavřeným vrcholem je start s .
- BFS otvírá na řádku (11) **všechny** nové sousedy dříve otevřených vrcholů.
- ($\supseteq$) Vrchol tedy bude otevřen (a později pak uzavřen) pouze tehdy, když do něho ze startu s vede posloupnost sousedních hran.
- ($\subseteq$) Naopak, existuje-li do vrcholu nějaká posloupnost sousedních hran z s , bude v průběhu algoritmu někdy otevřen.



Každý uzavřený vrchol patří do právě jedné hladiny H_i . Proto:

Hladiny H_i , $i = 0, \dots, h$, tvoří rozklad množiny uzavřených vrcholů.

Důkaz správnosti algoritmu BFS II

Vlastnost 2: Pro všechny uzavřené vrcholy v platí $D[v] = d(s, v) =$ délka nejkratší cesty ze startu s do vrcholu v .

Důkaz.

- Triviálně platí ve fázi F_0 . Uvažujme fázi F_i , $i > 0$.
- Pokud $v \in H_i$, pak $D[v] = i$ a tedy z s do v musí existovat cesta o i hranách. Proto je vzdálenost $d(s, v) \leq i$.
- Ukážeme nyní, že nemůže existovat s - v -cesta kratší než i . Pro spor, necht' existuje cesta P z s do v délky j , kde $j < i$.
- Protože $j < i$, musí dle holubníkového principu na P existovat dva sousední vrcholy w a w' a dvě nesousední hladiny H_k a H_ℓ (tedy $|k - \ell| > 1$) tak, že $w \in H_k$ a $w' \in H_\ell$.
- To však není možné, neboť dva sousední vrcholy na P nutně musí ležet ve dvou sousedních hladinách. Spor.
- Proto $d(s, v) = i = D[v]$. □

Koza, vlk a zelí

Farmář se ocitl vedouce s sebou kozu, vlka a nesouce velkou hlávku zelí na břehu řeky, kterou potřeboval překonat. Byla zde naštěstí přivázána loďka. Do té se ovšem vešel jen muž a buď koza, zelí nebo vlk.

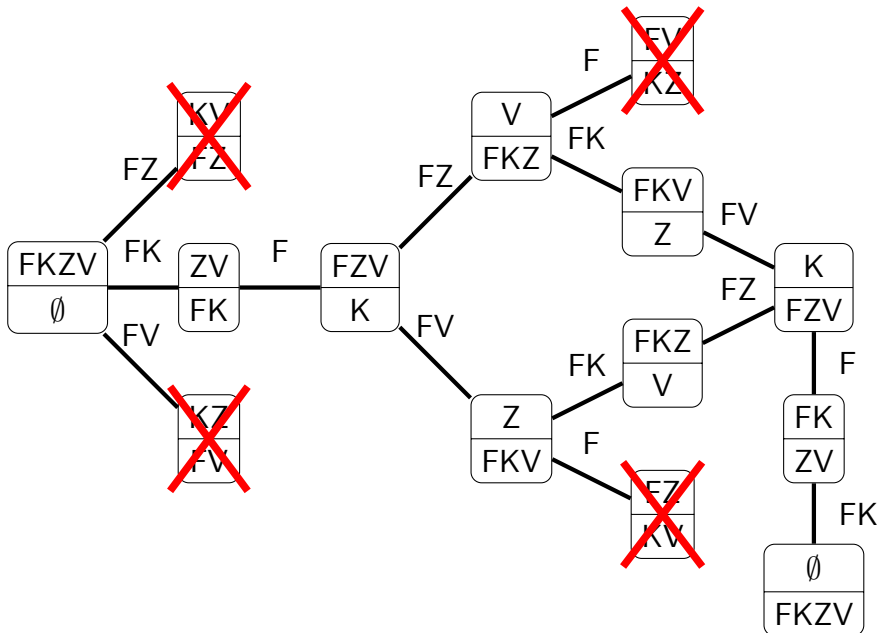
Může samozřejmě převézt vše postupně, problém ale je, že

- ① když nechá kozu o samotě se zelím, sežere jej a
- ② když nechá vlka o samotě s kozou, sežere jí.

Jak má farmář vše převézt na druhou stranu, aniž by koza či zelí přišly k úhoně?

- Pomůže mu samosebou graf.
- Vrcholy budou „konfigurace břehů řeky“ (celkem 2^4).
 - ▶ Loďka vždy u farmáře.
- Hrany budou mezi dvěma konfiguracemi, pokud se lze „napřímo“ dostat z jedné do druhé.

Koza, vlk a zelí

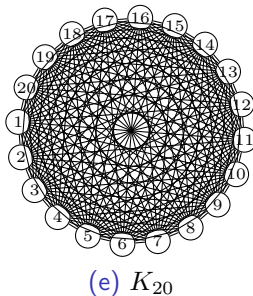
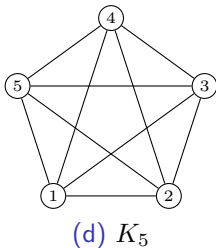
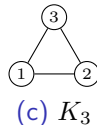
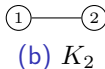


Úplný graf K_n („klika“)

Definice

Nechť $n \geq 1$.

Úplný graf na n vrcholech K_n je graf $(V, \binom{V}{2})$, kde $|V| = n$.

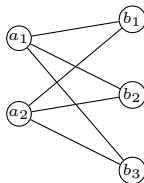


Úplný bipartitní graf K_{n_1, n_2}

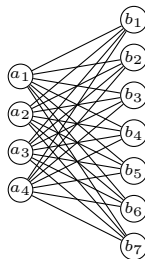
Definice

Nechť $n_1 \geq 1$ a $n_2 \geq 1$.

Úplný bipartitní graf K_{n_1, n_2} tvořený dvěma partitami o n_1 a n_2 vrcholech je graf $(A \cup B, \{\{a, b\} \mid a \in A, b \in B\})$, kde $A \cap B = \emptyset$, $|A| = n_1$ a $|B| = n_2$.



(a) $K_{2,3}$



(b) $K_{4,7}$

Všimněte si vztahu mezi velikostí partit a počtem hran: $|E| = n_1 \cdot n_2$
(Každý vrchol vlevo má hranu ke každému vpravo.)

Cesta P_n

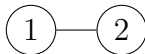
Definice

Nechť $n \geq 1$.

Cesta s n vrcholy (s $n - 1$ hranami) P_n je graf $(\{1, \dots, n\}, \{\{i, i + 1\} \mid i \in \{1, \dots, n - 1\}\})$.



(a) P_1 nebo také K_1



(b) P_2 nebo také K_2



(c) P_3 nebo také $K_{1,2}$



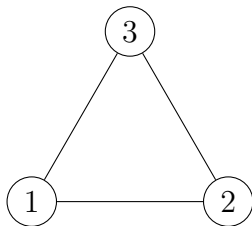
(d) P_5

Kružnice C_n

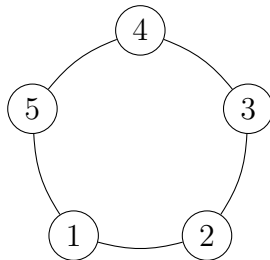
Definice

Nechť $n \geq 3$.

Kružnice délky n (s n vrcholy) C_n je graf $(\{1, \dots, n\}, \{\{i, i+1\} \mid i \in \{1, \dots, n-1\}\} \cup \{\{n, 1\}\})$.



(a) C_3 nebo také K_3



(b) C_5

Doplňěk grafu

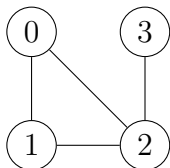
Definice

Doplňěk $\overline{G}$ grafu $G = (V, E)$ je graf $(V, \binom{V}{2} \setminus E)$.

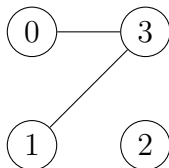
Příklad:

Nechť $G = (\{0, 1, 2, 3\}, \{\{0, 1\}, \{0, 2\}, \{1, 2\}, \{2, 3\}\})$.

Pak $\overline{G} = (\{0, 1, 2, 3\}, \{\{0, 3\}, \{1, 3\}\})$.



(a) Graf G



(b) Graf $\overline{G}$

Okolí, stupně

Definice

Nechť $G = (V, E)$ je graf a $v \in V$ jeho vrchol.

- Symbolem $\deg_G(v)$ označíme počet hran grafu G obsahujících vrchol v .

Toto číslo nazveme **stupněm** vrcholu v v grafu G .

- Symbolem $N_G(v)$ označíme množinu všech sousedů vrcholu v v grafu G .

Tuto množinu nazveme **(otevřené) okolí** vrcholu v v grafu G .

- Množinu $N_G[v] = N_G(v) \cup \{v\}$ nazveme **uzavřeným okolím** vrcholu v v grafu G .

- Platí tedy $\deg_G(v) = |N_G(v)|$.
- Namísto $\deg_G(v)$ budeme psát $\deg(v)$, pokud je z kontextu jasné, který graf máme na mysli.

Regulární graf, izolovaný vrchol

Definice

Graf je **r -regulární**, pokud stupeň každého jeho vrcholu je r .
Graf je **regulární**, pokud je r -regulární pro nějaké r .

Definice

Vrchol stupně 0 nazveme **izolovaný** (nemá žádné sousedy).

K zamyšlení:

- Jak vypadá 0-regulární graf o n vrcholech?
- Jak vypadá 1-regulární graf o n vrcholech?
- Existuje 1-regulární graf s lichým počtem vrcholů?
- Jak vypadá 2-regulární graf o n vrcholech?
- Existuje 2-regulární graf s lichým počtem vrcholů?
- Kolik existuje neizomorfních 2-regulárních grafů o n vrcholech?

Princip sudosti

Věta (o principu sudosti)

Pro každý graf $G = (V, E)$ platí

$$\sum_{v \in V} \deg_G(v) = 2|E|.$$

Důkaz.

Posčítáme-li stupně všech vrcholů, započítáme každou hranu přesně dvakrát, což dává součet $2|E|$.

$$\begin{aligned} \sum_{v \in V} \deg_G(v) &= \sum_{v \in V} |N_G(v)| \\ &= \sum_{v \in V} |\{u \in V \mid \{u, v\} \in E\}| = \\ \sum_{v \in V} |\{e \in E \mid v \in e\}| &= \sum_{v \in V} \sum_{e \in E | v \in e} 1 = \sum_{e \in E} \sum_{v \in e} 1 = \sum_{e \in E} 2 = 2|E| \end{aligned}$$



Princip sudosti

Věta o principu sudosti má řadu důsledků. Například:

Důsledek

- 1 V každém grafu je počet vrcholů lichého stupně sudý.
- 2 Každý regulární graf lichého stupně musí mít sudý počet vrcholů.

Podgraf

Definice

- Graf H je **podgrafem** grafu G , když $V(H) \subseteq V(G)$ a $E(H) \subseteq E(G)$.
- Graf H je **indukovaným podgrafem** grafu G , když $V(H) \subseteq V(G)$ a $E(H) = E(G) \cap \binom{V(H)}{2}$.
- Je-li $G = (V, E)$ a $V' \subseteq V$, pak $G[V']$ označuje graf s množinou vrcholů V' a množinou hran $E(G) \cap \binom{V'}{2}$.

Říkáme, že $G[V']$ je **podgraf indukovaný množinou vrcholů V'** .

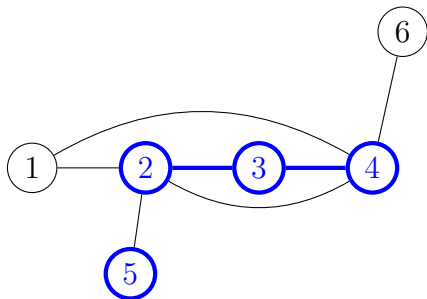
Graf $G[V \setminus V']$ budeme zapisovat zkráceně $G - V'$.

A speciálně, pokud $V' = \{v\}$, pak píšeme $G - v$ místo $G - \{v\}$.

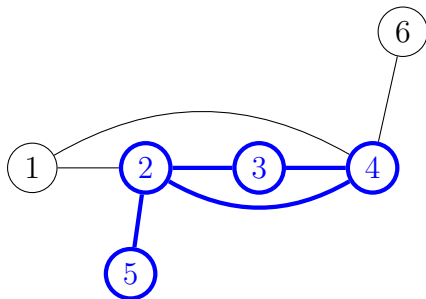
Definice

Podgraf úplného bipartitního grafu je **bipartitní graf**.

Podgraf - příklady

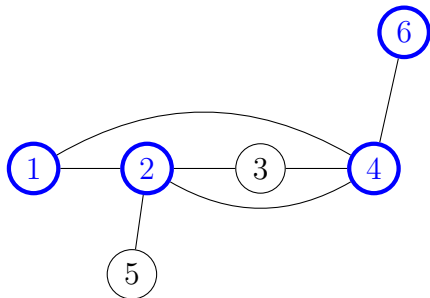


(a) Podgraf

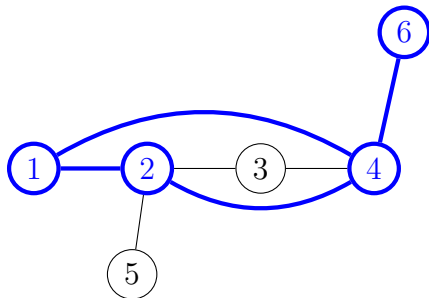


(b) Indukovaný podgraf na stejné množině vrcholů jako (a)

Podgraf - příklady



(c) Množina V'

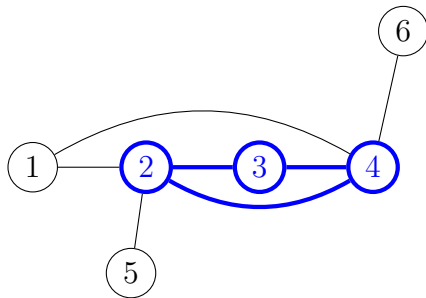


(d) Podgraf $G[V']$ indukovaný množinou V'

Klika

Definice

Klika v grafu G je podmnožina vrcholů, z nichž každé dva jsou sousední (čili spojeny hranou).



Orientovaný graf

Definice

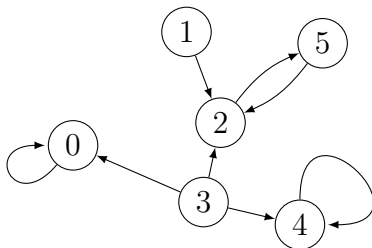
Orientovaný graf G je uspořádaná dvojice (V, E) , kde

- V je neprázdná konečná množina **vrcholů** a
- E je množina **orientovaných hran** (zobrazujeme jako **šipky**).

Orientovaná hrana $(u, v) \in E$ je **uspořádaná** dvojice vrcholů $u, v \in V$.

Říkáme, že u je **předchůdce** v a v je **následník** u .

Říkáme, že orientovaná hrana (u, u) je **smyčka**.



Algoritmus **BFS**(orientovaný graf G , vrchol s):

(1) Pro každý vrchol $v \in V(G)$:

(2) stav[v] := nenalezený

(3) D[v] := P[v] := undef

(4) Q := fronta obsahující jediný vrchol s

(5) stav[s] := otevřený

(6) D[s] := 0, P[s] := $\perp$

(7) Dokud je fronta Q neprázdná:

(8) Odeber z Q první vrchol, nechť to je v

(9) Pro všechny následníky w vrcholu v :

(10) Pokud stav[w] = nenalezený:

(11) stav[w] := otevřený

(12) D[w] := D[v] + 1

(13) P[w] := v

(14) Přidej w na konec fronty Q

(15) stav[v] := uzavřený

(16) Vrať (D, P)

Příklad modifikace pro BFS

Vstup

Neorientovaný graf $G = (V, E)$, vrchol $s \in V$. Rozdělení hran E na množiny červených R a modrých B hran (tj. $E = R \cup B$ a $R \cap B = \emptyset$).

Cestu z s do v v grafu G nazveme **přípustnou**, pokud používá nanejvýš jednu modrou hranu.

Výstup

- Pole vzdáleností D takové, že

$$D[v] = \begin{cases} k & k \text{ je délka nejkratší přípustné } s\text{-}v\text{-cesty v } G \\ \text{undef.} & \text{neexistuje žádná přípustná } s\text{-}v\text{-cesta v } G \end{cases}$$

- Pole předchůdců P pro (případné) přípustné nejkratší cesty v G .

Příklad, k čemu lze nejkratší cesty v grafu použít

Netriviální příklad (který nevypadá jako grafová úloha):

Úloha

- Je dáno přirozené číslo $K > 1$.
- Chceme najít přirozené číslo $N \geq 1$ s **nejmenším počtem cifer** (tj. co nejkratší číslo), které
 - ▶ má v **desítkovém** zápise pouze číslice 0 a 1 a
 - ▶ je **dělitelné** K ,případně oznámit, že takové číslo neexistuje.

Protože $N \geq 1$, je zřejmé, že N musí začínat číslicí 1.

Nejkratší číslo v dekadické soustavě s binárními číslicemi dělitelné

- číslem $K = 3$ je $N = 111$
- číslem $K = 4$ je $N = 100$
- číslem $K = 5$ je $N = 10$
- číslem $K = 6$ je $N = 1110$

Co bychom po dnešku měli znát

Témata probraná na dnešní přednášce:

- (ne)orientovaný graf, vrchol, hrana
- sousedé, incidence
- cesta, kružnice
- cesta v grafu a její délka
- vzdálenost v (neohodnoceném) grafu
- úplný graf, úplný bipartitní
- nalezení nejkratších cest z jednoho vrcholu do všech ostatních v (neohodnoceném) grafu – prohledávání do šířky
- uzavřené a otevřené okolí vrcholu, stupeň vrcholu, izolovaný vrchol
- regulární graf
- podgrafy, indukované podgrafy